

Edinaldo Matias dos Santos Junior
André Luis Ferreira de Oliveira

**Otimização de Rotas Internas no Senac
Considerando a Qualidade do Wi-Fi: Uma
Comparação entre Algoritmo de Grafos e
Inteligência Artificial**

São Paulo
2026

Edinaldo Matias dos Santos Junior
André Luis Ferreira de Oliveira

Otimização de Rotas Internas no Senac Considerando a Qualidade do Wi-Fi: Uma Comparação entre Algoritmo de Grafos e Inteligência Artificial

Proposta de Trabalho de Conclusão de Curso
apresentado ao SENAC Santo Amaro como
requisito parcial para aprovação na disciplina
de TCC1 do curso de Ciência da Computação.

SENAC – Serviço Nacional de Aprendizagem Comercial
Unidade Santo Amaro
Curso de Ciência da Computação

Orientador: Prof. Thyago Conchado Quintas

São Paulo
2026

Resumo

Este documento apresenta a proposta inicial do Trabalho de Conclusão de Curso (TCC1) desenvolvido no Centro Universitário SENAC Santo Amaro. O objetivo deste trabalho é comparar a eficácia do algoritmo determinístico de Dijkstra com um modelo de Inteligência Artificial (IA) baseado em Aprendizado por Reforço (RL) na otimização de rotas internas, utilizando a estabilidade do sinal Wi-Fi como variável de custo dinâmico. A justificativa para este estudo baseia-se na ineficácia do Sistema de Posicionamento Global (GPS) em espaços fechados e na vulnerabilidade de abordagens clássicas de roteamento estático frente às flutuações estocásticas de sinais sem fio em Ambientes Dinâmicos (DE). A revisão bibliográfica abrange as limitações dos sistemas de posicionamento *indoor*, a teoria dos grafos aplicada à descoberta de caminhos, a métrica Received Signal Strength Indicator (RSSI) e os fundamentos do roteamento adaptativo (*Q-Routing*). Como solução proposta, pretende-se inicialmente modelar os corredores do Prédio Acadêmico I da instituição como uma rede de grafos, compilar um banco de dados empírico das variações do sinal Wi-Fi através da plataforma *IndoorAtlas* e submeter ambos os algoritmos a testes comparativos sob duas condições: cenários de controle (representando estabilidade de conexão) e cenários de estresse (simulando quedas abruptas de sinal e congestionamento de rede), provando a capacidade da IA em priorizar a conectividade em detrimento da distância física. Este trabalho está organizado em capítulos que apresentam a introdução ao tema, a revisão bibliográfica que fundamenta a pesquisa, o desenvolvimento com a solução proposta e o cronograma de trabalho, seguidos das referências bibliográficas.

Palavras-chave: Navegação *Indoor*. Teoria dos Grafos. Dijkstra. *Q-Learning*. RSSI.

Lista de ilustrações

Figura 1 – Expressão dos decibéis por miliwatt	12
Figura 2 – Fatores Dinâmicos	13
Figura 3 – Grafo ponderado não direcionado para execução do algoritmo de Dijkstra	14
Figura 4 – Diagrama de pesquisa bibliografica	16
Figura 5 – Critérios de inclusão	17
Figura 6 – Critérios de exclusão	17
Figura 7 – Fluxo do Ambiente Computacional de Simulação	32
Figura 8 – Cronograma de atividades – Diagrama de Gantt	33

Lista de tabelas

Tabela 1 – Comparação entre trabalhos relacionados e a proposta atual	28
---	----

Lista de abreviaturas e siglas

A*	Algoritmo de busca informada heurística (<i>A-star</i>)
API	<i>Application Programming Interface</i> (Interface de Programação de Aplicações)
BSSID	<i>Basic Service Set Identifier</i> (Identificador único de ponto de acesso Wi-Fi)
dBm	Decibéis por miliwatt
DE	<i>Dynamic Environment</i> (Ambiente Dinâmico)
DL	<i>Deep Learning</i> (Aprendizado Profundo)
GPS	Sistema de Posicionamento Global
IA	Inteligência Artificial
IEEE	<i>Institute of Electrical and Electronics Engineers</i>
IPS	<i>Indoor Positioning System</i> (Sistema de Posicionamento <i>Indoor</i>)
JPEG	<i>Joint Photographic Experts Group</i> (formato de imagem digital)
MAC	<i>Media Access Control</i> (endereço físico único de dispositivos de rede)
MCDM	<i>Multi-Criteria Decision Making</i> (Tomada de Decisão por Múltiplos Critérios)
MDP	<i>Markov Decision Process</i> (Processo de Decisão de Markov)
ML	<i>Machine Learning</i> (Aprendizado de Máquina)
PNG	<i>Portable Network Graphics</i> (formato de imagem digital)
REST API	<i>Representational State Transfer Application Programming Interface</i>
RL	<i>Reinforcement Learning</i> (Aprendizado por Reforço)
RSL	Revisão Sistemática da Literatura
RSSI	<i>Received Signal Strength Indicator</i> (Indicador de Força do Sinal Recebido)
TD	<i>Temporal Difference</i> (Diferença Temporal)

TCC1	Trabalho de Conclusão de Curso 1
TCC2	Trabalho de Conclusão de Curso 2
VISSIM	<i>Verkehr in Städten – SIMulationsmodell</i> (software de microsimulação de tráfego)
WSDA	<i>Weighted Sum-Dijkstra's Algorithm</i> (Algoritmo de Soma Ponderada de Dijkstra)

Sumário

1	INTRODUÇÃO	8
1.1	Contexto	8
1.2	Justificativa	9
1.3	Objetivo	11
1.3.1	Objetivo principal	11
1.3.2	Objetivos secundários	11
2	REVISÃO BIBLIOGRÁFICA	12
2.1	Referencial teórico	12
2.1.1	Navegação <i>Indoor</i> , Wi-Fi e a Métrica RSSI	12
2.1.2	Teoria dos Grafos e o Algoritmo de Dijkstra	13
2.1.3	Inteligência Artificial e Aprendizado por Reforço no Roteamento	14
2.1.4	Algoritmo Q-Learning, Roteamento Q e Tabela Q	15
2.2	Metodologia da pesquisa bibliográfica	16
2.3	Fundamentos	17
2.3.1	Modelagem Topológica e Plataforma <i>IndoorAtlas</i>	17
2.3.2	A Função de Custo: Distância e RSSI	19
2.3.3	Implementação do Algoritmo de Dijkstra	20
2.3.4	Implementação do Algoritmo <i>Q-Learning</i>	23
2.4	Trabalhos relacionados	25
2.4.1	Trabalho 1: Indoor Atlas Service as a Tool for Building an Interior Navigation System	26
2.4.2	Trabalho 2: Reinforcement learning based estimation of shortest paths in dynamically changing transportation networks	26
2.4.3	Trabalho 3: The Challenge of Dynamic Environments in Regard to RSSI-Based Indoor Wi-Fi Positioning	27
3	DESENVOLVIMENTO	29
3.1	Solução proposta	29
3.2	Cronograma de trabalho	32
	REFERÊNCIAS	34

1 Introdução

O sistema de posicionamento e navegação em ambientes externos (*outdoor*) consolidou-se como uma tecnologia altamente eficiente, fornecendo a localização precisa de objetos e usuários por meio de sinais de satélite, como o Sistema de Posicionamento Global (GPS). No entanto, esse método opera de forma eficaz apenas em áreas abertas com linha de visão direta. Quando o usuário adentra edifícios, os sinais sofrem interferência severa ou bloqueio completo por paredes e telhados, tornando o GPS ineficaz para navegação interna (CHIA et al., 2025; BARROSO; BRITO, 2025).

Para solucionar a navegação nesses espaços fechados, Štancel et al. (2021) afirmam que o mundo real pode ser mapeado computacionalmente através de um grafo de vértices e arestas. A viabilidade dessa modelagem estrutural especificamente para os corredores do Senac já foi comprovada empiricamente por Barroso e Brito (2025). A partir dessa modelagem, algoritmos clássicos de teoria dos grafos, como Dijkstra e Algoritmo de busca informada heurística (A^*), são muito bem-sucedidos ao definir a menor rota para redes com custos de arestas estáticos. Entretanto, no contexto prático real, os custos das arestas mudam dinamicamente.

Sendo assim, essas abordagens tradicionais apresentam várias limitações em cenários onde os custos de ligação mudam dinamicamente — como a evitar congestionamentos em redes de transporte (PHAM et al., 2025) ou, no contexto deste trabalho, a manutenção de uma conectividade Wi-Fi estável em ambientes indoor.

Para Pham et al. (2025), algoritmos baseados em aprendizado por reforço (RL) computam ótimos resultados quando tratamos de redes que mudam dinamicamente. Dessa forma, este estudo propõe realizar um comparativo prático no Centro Universitário Senac entre o algoritmo determinístico de Dijkstra e um modelo de Inteligência Artificial (IA) baseado em RL, utilizando como pesos das arestas a distância física (metros) e a qualidade do sinal baseada no Received Signal Strength Indicator (RSSI).

1.1 Contexto

O desenvolvimento de sistemas de posicionamento e navegação revolucionou a mobilidade e a interação espacial na sociedade contemporânea. Em ambientes externos (*outdoor*), tecnologias baseadas em constelações de satélites, como o GPS, consolidaram-se como soluções altamente eficientes para o rastreamento preciso de usuários. No entanto, a eficácia desse método restringe-se a áreas abertas com linha de visão direta. Quando o indivíduo adentra edifícios e complexos arquitetônicos fechados, os sinais de rádio sofrem interferência severa ou bloqueio total por estruturas físicas, como paredes e lajes, tornando o GPS ineficaz para a navegação interna (*indoor*) (CHIA et al., 2025; BARROSO; BRITO, 2025).

Para viabilizar o deslocamento nesses espaços fechados, a literatura e a indústria têm recorrido a infraestruturas locais preexistentes, com destaque para a utilização de redes sem fio (Wi-Fi). Computacionalmente, o ambiente físico de uma instituição pode ser mapeado e abstraído por meio de uma estrutura de grafos, composta por vértices e arestas que representam os cruzamentos e os corredores (ŠTANCEL et al., 2021). Para quantificar o peso ou custo dessas arestas visando a manutenção da conectividade, a métrica de RSSI, expressa em decibéis por miliwatt (dBm), estabeleceu-se como um padrão de medição.

Entretanto, o domínio físico em que esse sistema opera caracteriza-se por ser um Ambiente Dinâmico (DE - *Dynamic Environment*). Em instituições de grande porte, o fluxo estocástico de pessoas, a movimentação de mobiliário, o ruído eletromagnético e a interferência de múltiplos caminhos (*multipath interference*) causam flutuações constantes na propagação do sinal. Consequentemente, a força do sinal Wi-Fi torna-se altamente vulnerável à atenuação, resultando em uma métrica de custo dinâmico e instável (CHIA et al., 2025).

A existência desse fenômeno dinâmico expõe uma lacuna crítica nas abordagens computacionais de roteamento. Algoritmos clássicos da teoria dos grafos, como o método de Dijkstra, são amplamente utilizados e comprovadamente ótimos para definir a menor rota em redes onde os custos das arestas são estáticos. Contudo, em um cenário prático e real, as abordagens determinísticas tradicionais apresentam limitações severas, pois são incapazes de adaptar o roteamento de forma reativa a custos de ligação que mudam em tempo real (PHAM et al., 2025).

A incapacidade de resolver essa assimetria entre algoritmos estáticos e ambientes dinâmicos (DE) afeta diretamente os usuários finais que dependem de navegação assistida e conectividade ininterrupta em amplas instalações. A principal consequência de ignorar essas flutuações é a geração de rotas que, apesar de geograficamente mais curtas, direcionam o indivíduo para "zonas de sombra" ou áreas de alta atenuação de rede.

Diante desse cenário, o estudo de métodos adaptativos evidencia sua relevância. Para a sociedade, garante uma experiência de navegação contínua e conectada. Para a Ciência da Computação, preenche a lacuna metodológica identificada na literatura atual, investigando como técnicas emergentes de IA — especificamente o RL, capaz de lidar com a dinamicidade de redes de forma autônoma — podem superar os limites dos algoritmos estáticos no roteamento em ambientes complexos (PHAM et al., 2025).

1.2 Justificativa

A motivação para o desenvolvimento deste trabalho fundamenta-se na necessidade de adequar os sistemas de roteamento *indoor* à realidade estocástica das infraestruturas de rede modernas. Historicamente, o problema de caminhos mínimos tem sido tratado sob uma ótica estática, o que se mostra insuficiente frente às flutuações inerentes a ambientes

com alta circulação. Diante disso, a pertinência desta pesquisa sustenta-se sob três pilares fundamentais: a identificação de uma lacuna na literatura, a sua relevância acadêmica e a sua aplicabilidade prática.

- **Relevância acadêmica:** Este trabalho contribui diretamente para o avanço do conhecimento na intersecção entre a Teoria dos Grafos e a IA. Ao propor um comparativo empírico entre um algoritmo determinístico clássico (Dijkstra) e um modelo adaptativo (*Q-Learning*) sob a aplicação de princípios de engenharia de caos — prática metodológica que consiste na injeção intencional de anomalias e falhas controladas (como a degradação abrupta do sinal Wi-Fi) para testar a resiliência do sistema —, a pesquisa não se limita a revisar conceitos existentes. Ela fornece dados práticos sobre o comportamento, a convergência e a resiliência de modelos de RL na mitigação de flutuações do sinal RSSI. Essa abordagem eleva a discussão acadêmica ao demonstrar como a variabilidade do ambiente pode ser tratada matematicamente não como um ruído impeditivo, mas como uma variável de custo penalizadora na função de recompensa de um agente inteligente.
- **Relevância prática/profissional:** No âmbito aplicado, os benefícios gerados por esta solução impactam diretamente a experiência dos usuários e a gestão de infraestrutura de instituições de grande porte. O desenvolvimento de um sistema de roteamento inteligente permitirá que transeuntes — sejam alunos, funcionários ou visitantes do Centro Universitário Senac — desloquem-se pelas dependências do campus de maneira otimizada. A solução proposta garante não apenas a indicação do caminho fisicamente viável, mas, primordialmente, a manutenção da conectividade com a internet durante todo o trajeto. Evitar a transição por "zonas de sombra" previne a interrupção de aplicações em tempo real, chamadas de vídeo e transmissões de dados, agregando um valor operacional direto para a comunidade e servindo como um modelo escalável para hospitais, aeroportos e polos corporativos.
- **Lacuna identificada:** A revisão da literatura atual revela uma escassez significativa de estudos que abordem o roteamento físico sob a perspectiva dos DE. Conforme apontado por [Chia et al. \(2025\)](#), a grande maioria das pesquisas sobre posicionamento *indoor* baseado em Wi-Fi ignora as flutuações contínuas do ambiente, assumindo premissas de estabilidade que não se verificam no mundo real. Paralelamente, embora os métodos de RL sejam amplamente consolidados para o cálculo de rotas dinâmicas em redes lógicas de computadores, o seu potencial para redes de transporte físico e navegação humana começou a atrair atenção apenas recentemente ([PHAM et al., 2025](#)). Desse modo, carece-se de estudos empíricos que unam essas técnicas de Inteligência Artificial à otimização de caminhos guiados por fatores dinâmicos de infraestrutura.

1.3 Objetivo

Esta seção define as metas estabelecidas para a resolução do problema de roteamento *indoor* em ambientes dinâmicos, apresentando o propósito central da pesquisa e as etapas metodológicas necessárias para sua execução.

1.3.1 Objetivo principal

Comparar a eficácia do algoritmo determinístico de Dijkstra com um modelo de Inteligência Artificial baseado em Aprendizado por Reforço (*Q-Learning*) na otimização de rotas internas no Centro Universitário Senac, utilizando a qualidade e a estabilidade do sinal Wi-Fi (RSSI) como variável de custo dinâmico.

1.3.2 Objetivos secundários

Os objetivos secundários representam os desdobramentos metodológicos e práticos necessários para o alcance da meta principal:

1. Realizar o mapeamento topológico da planta baixa do Centro Universitário Senac, convertendo os corredores e cruzamentos da instituição em uma rede de grafos navegável;
2. Coletar e compilar um banco de dados empírico contendo as aferições reais da potência do sinal Wi-Fi (RSSI) no ambiente estudado. O *dataset* de RSSI será coletado ao longo de **três semanas**, em **três dias por semana**, com medições realizadas em **dois horários distintos** por dia: entre 18h00 e 19h00 (antes do início das aulas, com baixo fluxo de pessoas) e entre 20h50 e 21h20 (durante o intervalo, com alto fluxo de pessoas)
3. Implementar o algoritmo clássico de Dijkstra para o roteamento na rede, adotando a métrica de distância física combinada com a média estática do RSSI como peso fixo das arestas;
4. Implementar o algoritmo de *Q-Learning*, realizando a modelagem matemática do ambiente (estados, ações e função de custo/recompensa) para que o agente autônomo aprenda interativamente a contornar rotas de baixa conectividade;
5. Avaliar e comparar o desempenho de ambas as abordagens computacionais em cenários estáticos (ordem) e dinâmicos (caos), com base nos critérios de completude, grau de otimização, complexidade de tempo e complexidade de espaço.

2 Revisão Bibliográfica

Este capítulo apresenta a fundamentação teórica que sustenta o desenvolvimento desta pesquisa. A revisão está organizada em conceitos estruturais que abordam os desafios da navegação *indoor*, o comportamento estocástico da métrica RSSI, os fundamentos da Teoria dos Grafos e os preceitos da Inteligência Artificial aplicados ao roteamento dinâmico.

2.1 Referencial teórico

2.1.1 Navegação *Indoor*, Wi-Fi e a Métrica RSSI

A complexidade de localizar e guiar usuários em ambientes fechados difere drasticamente da navegação *outdoor*. Enquanto espaços abertos dependem do GPS, a infraestrutura *indoor* exige tecnologias locais devido à severa atenuação que os sinais de satélite sofrem ao atravessar barreiras arquitetônicas. Para contornar esse desafio tecnológico, [Štancel et al. \(2021\)](#) demonstram que o posicionamento indoor pode ser viabilizado sem infraestrutura adicional de hardware, utilizando sensores já presentes nos smartphones, permitindo o posicionamento em aeroportos, garagens e campus universitários sem a necessidade de infraestrutura de hardware adicional no dispositivo do usuário.

A principal métrica extraída dessa rede é o *Received Signal Strength Indicator* (RSSI), que quantifica a força do sinal recebido por um dispositivo móvel. O valor do RSSI é expresso em dBm e é comumente lido em espectros negativos, onde valores mais próximos de zero indicam maior qualidade de conexão.

Figura 1 – Expressão dos decibéis por miliwatt

Força do Sinal	Classificação
>-50 dBm	Excelente
-50 dBm a -60 dBm	Bom
-60 dBm a -70 dBm	Razoável
< -70 dBm	Fraco

Fonte: Elaborado pelo autor, com base em [Chia et al. \(2025\)](#)

Contudo, a literatura aponta que a estabilidade dessa métrica é altamente vulnerável. Segundo [Chia et al. \(2025\)](#), espaços internos fechados e com alto fluxo de usuários são classificados como Ambientes Dinâmicos. Nesses cenários, fatores físicos — como o bloqueio do sinal pela circulação de massas humanas — e de rede — como interferência de múltiplos

caminhos (*multipath interference*) — geram flutuações constantes e quedas abruptas no nível do RSSI.

Figura 2 – Fatores Dinâmicos



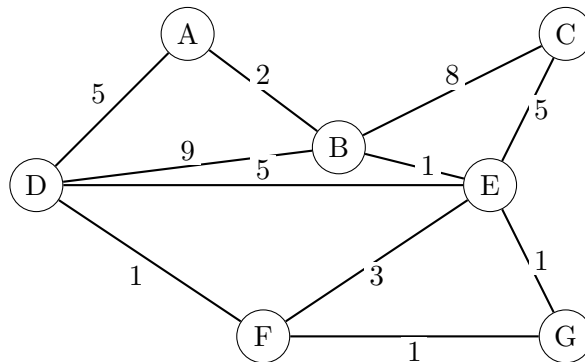
Fonte: Traduzido pelo autor, com base em [Chia et al. \(2025\)](#)

2.1.2 Teoria dos Grafos e o Algoritmo de Dijkstra

Para que a navegação seja computacionalmente viável, o ambiente físico deve ser abstraído em uma estrutura matemática de grafos. Grafos constituem estruturas matemáticas altamente versáteis cuja função principal é modelar sistemas compostos por entidades e pelas relações entre elas ([GOLDBARG; GOLDBARG, 2012](#)). Em termos formais, um grafo é formado por um conjunto de pontos, denominados vértices (ou nós), e pelas ligações que os conectam, chamadas de arestas (quando não possuem orientação) ou arcos (quando possuem direção) ([GOLDBARG; GOLDBARG, 2012](#)). Na modelagem de roteamento espacial, utilizam-se comumente grafos simples e não direcionados, garantindo que a conexão entre dois vértices seja bidirecional — permitindo o deslocamento de ida e volta — e que não existam laços conectando um vértice a ele mesmo ([GOLDBARG; GOLDBARG, 2012](#)). A viabilidade dessa modelagem especificamente para os corredores do Senac foi comprovada empiricamente por [Barroso e Brito \(2025\)](#), que representaram salas, escadas e corredores como vértices e suas conexões físicas como arestas de um grafo não direcionado. A partir da noção de vizinhança, que ocorre quando existe uma aresta direta entre dois nós, algoritmos computacionais conseguem definir sequências de passos conhecidas como caminhos. Dessa forma, corredores e salas são representados como vértices (nós), e os caminhos que os conectam são representados como arestas (ligações), atribuindo-se um peso matemático a cada trajeto ([ŠTANCEL et al., 2021](#)).

Nesse modelo, o algoritmo de Dijkstra, proposto originalmente em 1959, é o padrão-ouro para a determinação de caminhos mínimos em grafos ponderados ([CORMEN et al., 2009](#)). O algoritmo expande os nós a partir de um vértice de origem, acumulando e validando os menores custos até alcançar os demais vértices do grafo, sob a condição de que os pesos das arestas sejam não negativos.

Figura 3 – Grafo ponderado não direcionado para execução do algoritmo de Dijkstra



Fonte: Adaptado de (Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira, 2021) pelo autor (2026)

Apesar de sua excelência e completude matemática, o algoritmo de Dijkstra foi concebido para redes determinísticas, onde o custo da ligação é estático (como a distância em metros). No contexto prático de um Ambiente Dinâmico, onde o peso da aresta inclui a qualidade variável do sinal Wi-Fi, o custo muda continuamente. Para manter a rota ótima frente a uma queda de sinal, o algoritmo tradicional demandaria recálculos iterativos completos da árvore de caminhos, gerando uma sobrecarga de processamento incompatível com o redirecionamento contínuo em tempo real (PHAM et al., 2025).

2.1.3 Inteligência Artificial e Aprendizado por Reforço no Roteamento

Para solucionar a incapacidade dos métodos clássicos em lidar com o "caos" dos ambientes dinâmicos, a pesquisa recorre à IA, com foco estrito no Aprendizado por Reforço. É imperativo distinguir essa abordagem de outras ramificações da IA, como o Aprendizado Profundo (DL). Enquanto as redes neurais são frequentemente usadas no problema de reconhecimento de posições passivas (*Fingerprinting*), o RL é projetado especificamente para a tomada de decisão sequencial e ativa em ambientes com incertezas (SUTTON; BARTO, 2018).

Conforme a literatura recente sobre estimação de rotas, os métodos baseados em RL não necessitam conhecer os custos de ligação de antemão para iniciar a operação em uma rede que muda dinamicamente (PHAM et al., 2025). O sistema computacional atua como um agente que mapeia e aprende os caminhos ótimos estritamente por meio de tentativa, erro e interação contínua com o ambiente. Em cenários estocásticos, a aplicação de RL permite que a navegação se adapte em tempo real, mitigando a variabilidade do ambiente sem o alto custo computacional do recálculo estático.

2.1.4 Algoritmo Q-Learning, Roteamento Q e Tabela Q

Dentro do paradigma do Aprendizado por Reforço, o modelo matemático central deste trabalho é o algoritmo *Q-Learning*. Proposto originalmente por [Watkins \(1989\)](#) e amplamente formalizado por [Sutton e Barto \(2018\)](#), trata-se de um algoritmo livre de modelo (*model-free*), que dispensa o conhecimento prévio das probabilidades de transição do ambiente. Ele descobre a política ótima de forma iterativa, avaliando a função de Qualidade (o "Q" de *Quality*) de tomar uma determinada ação (a) partindo de um determinado estado (s).

Quando o *Q-Learning* é aplicado especificamente para a descoberta de rotas em estruturas de grafos ou redes de transporte, a literatura denomina a técnica como Roteamento Q (*Q-Routing*) ([BOYAN; LITTMAN, 1993](#)). O núcleo matemático desse algoritmo reside em uma estrutura matricial conhecida como Tabela Q (*Q-Table*). Nesta matriz, as linhas representam os nós do grafo (corredores) e as colunas representam as transições possíveis. Em cada célula, armazena-se um valor $Q(s, a)$, que reflete a expectativa de custo/recompensa cumulativa daquele trajeto. Neste trabalho, essa Função de Custo integra a atenuação do sinal (RSSI) e a distância física.

Conforme detalhado por [Pham et al. \(2025\)](#), o uso do Roteamento Q em redes onde os custos de ligação mudam dinamicamente destaca-se por sua arquitetura de execução dividida em duas etapas. Embora a pesquisa empírica do autor tenha sido validada em redes de transporte estocásticas, a mesma lógica algorítmica aplica-se à instabilidade do sinal Wi-Fi. Assim, a execução do modelo divide-se em duas fases operacionais distintas:

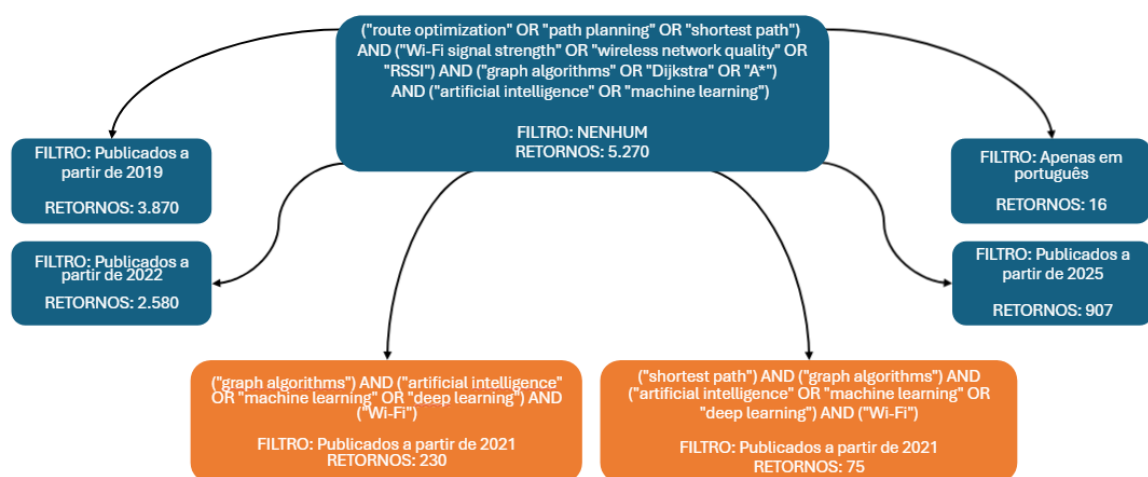
- **Fase *Offline* (Treinamento/Exploração):** Trata-se da etapa que demanda o tempo computacional necessário para o aprendizado do modelo. O agente percorre a rede em múltiplos episódios simulados, interagindo com o ambiente. Ao sofrer penalidades em caminhos com ruído ou quedas abruptas de sinal Wi-Fi, o algoritmo atualiza a Tabela Q, aprendendo os custos de ligação em mudança estritamente por meio da experiência.
- **Fase *Online* (Execução/Inferência):** Uma vez consolidado o aprendizado na Tabela Q, o roteamento real dinâmico ocorre de forma imediata. Segundo [Pham et al. \(2025\)](#), o Roteamento Q pode ser executado com uma complexidade de tempo tão igual quanto a do algoritmo de Dijkstra durante esta fase. A grande diferença apontada pelo autor reside na disponibilidade dos dados: enquanto o algoritmo de Dijkstra exige o conhecimento prévio e exato dos pesos das arestas (cenário de informação perfeita) para encontrar a rota, a Inteligência Artificial é capaz de computar o caminho ótimo sem conhecer os custos reais previamente (cenário de informação imperfeita), guiando-se exclusivamente por sua matriz de aprendizado.

2.2 Metodologia da pesquisa bibliográfica

A seguir são apresentados os principais tópicos que nortearam a condução desta pesquisa:

- **Bases de dados consultadas:** Google Scholar, IEEE Xplore;
- **Palavras-chave utilizadas:**
 - ("route optimization"OR "path planning"OR "shortest path") AND ("Wi-Fi signal strength"OR "wireless network quality"OR "RSSI") AND ("graph algorithms"OR "Dijkstra"OR "A*") AND ("artificial intelligence"OR "machine learning");
 - ("graph algorithms") AND ("artificial intelligence"OR "machine learning"OR "deep learning") AND ("Wi-Fi");
 - ("shortest path") AND ("graph algorithms") AND ("artificial intelligence"OR "machine learning"OR "deep learning") AND ("Wi-Fi").
- **CrITÉRIOS de inclusão e exclusão:** Os critérios de inclusão adotados foram: artigos que resolvem o mesmo problema central deste trabalho; estudos que descrevem detalhadamente a técnica utilizada e trazem métricas de desempenho. Como critérios de exclusão, foram descartados: trabalhos com tema diferente do problema estudado; estudos sem um método claro; e pesquisas com ausência de avaliação experimental;
- **Período de abrangência:** Últimos 5 anos (publicações a partir de 2021).

Figura 4 – Diagrama de pesquisa bibliografica



Fonte: Elaborado pelo autor (2026)

Figura 5 – Critérios de inclusão

Critérios de Inclusão
Resolve o mesmo problema central
Descreve técnica utilizada
Traz métricas
Está dentro do período definido (Dentro dos últimos 5 anos)

Fonte: Elaborado pelo autor (2026)

Figura 6 – Critérios de exclusão

Critérios de Exclusão
Tema diferente do problema estudado
Estudo sem método claro
Ausência de avaliação experimental

Fonte: Elaborado pelo autor (2026)

2.3 Fundamentos

Esta seção detalha as tecnologias, métricas e algoritmos específicos que serão empregados no desenvolvimento da solução de roteamento *indoor*. Diferentemente da base teórica geral, os tópicos a seguir descrevem a modelagem matemática e estrutural que sustentará a implementação prática do comparativo entre o método determinístico e a Inteligência Artificial.

2.3.1 Modelagem Topológica e Plataforma *IndoorAtlas*

Para que os algoritmos computacionais possam calcular rotas físicas, o ambiente real do Centro Universitário Senac precisa ser abstraído matematicamente em uma estrutura navegável. Para essa digitalização, será utilizada a plataforma *IndoorAtlas*, uma ferramenta especializada em serviços de posicionamento e mapeamento interno que dispensa a instalação de qualquer equipamento adicional na infraestrutura do edifício ([ŠTANCEL et al., 2021](#)).

Princípio de Funcionamento: Fusão de Dados Magnéticos e Wi-Fi

O *IndoorAtlas* opera por meio da fusão de duas fontes de dados complementares. A primeira é o campo magnético do edifício: conforme detalhado por [Štancel et al. \(2021\)](#),

estruturas modernas construídas com aço e materiais ferromagnéticos possuem impressões digitais magnéticas únicas em cada ponto do espaço, geradas pela interação do campo magnético da Terra com esses materiais. O sensor magnético do *smartphone* captura essas variações e as envia ao serviço, que as compara com um banco de dados de referência para estimar a posição do dispositivo.

A segunda fonte de dados é o sinal Wi-Fi. Conforme especificado na documentação oficial da plataforma, a Positioning *REST API* (*Application Programming Interface*) do IndoorAtlas opera lendo as varreduras Wi-Fi. do dispositivo, compostas por uma lista dos endereços Media Access Control (MAC) (*BSSIDs*) dos pontos de acesso detectados e os respectivos valores de força de sinal, expressos em RSSI (em dBm) (IndoorAtlas, 2025). Esses valores são enviados ao endpoint `/locate` da Application Programming Interface (API), que retorna as coordenadas geográficas, o número do andar e o raio de incerteza da posição calculada (IndoorAtlas, 2025). Na prática, isso significa que a cada varredura o sistema registra, para cada corredor, quais redes Wi-Fi estão visíveis e com qual intensidade de sinal, construindo o *dataset* empírico de RSSI que este trabalho utilizará como variável de custo dinâmico das arestas do grafo.

Mapeamento (*Offline*)

O processo de utilização da plataforma divide-se em duas fases. A primeira, denominada fase *offline*, é responsável pela construção do banco de dados de referência. Conforme descrito por Štancel et al. (2021), a planta baixa do edifício é adicionada ao serviço *IndoorAtlas FloorPlans* em formato PNG ou JPEG, com atribuição das coordenadas geográficas reais a cada ponto da planta. O pesquisador então percorre fisicamente os corredores com um *smartphone*, coletando simultaneamente as leituras do campo magnético e as varreduras de RSSI Wi-Fi em cada trecho. Esses dados são enviados à API e armazenados no banco de dados de impressões digitais (*fingerprint database*), a partir do qual o mapa de referência do edifício é construído (ŠTANCEL et al., 2021).

Para garantir a qualidade dos dados, Štancel et al. (2021) estabelecem um protocolo rigoroso de coleta: o dispositivo deve ser mantido verticalmente a pelo menos 50 cm das paredes, o coletor deve caminhar em ritmo constante e em linha reta, e a calibração do sensor deve ser realizada regularmente.

Em paralelo, a plataforma converte a planta baixa em um grafo $G = (V, A)$, onde os vértices (V) representam os cruzamentos e pontos de interesse dos corredores, e as arestas (A) representam os caminhos navegáveis entre eles. Cada vértice é definido por identificador, nome, latitude, longitude e andar; cada aresta é definida pelo nó de origem, nó de destino e a distância calculada pela fórmula de Haversine a partir das coordenadas geográficas reais (ŠTANCEL et al., 2021).

Aplicação no Presente Trabalho

No contexto desta pesquisa, a plataforma *IndoorAtlas* será utilizada exclusivamente em sua capacidade de mapeamento e coleta empírica, correspondente à fase *offline* descrita anteriormente. Não há, no escopo deste trabalho, a necessidade de posicionamento do usuário em tempo real: a origem e o destino de cada simulação serão definidos manualmente pelo pesquisador como nós do grafo.

A contribuição da plataforma restringe-se, portanto, a duas entregas fundamentais: a topologia navegável do edifício, representada como um grafo de vértices e arestas com coordenadas geográficas reais (ŠTANČEL et al., 2021), e o *dataset* empírico de RSSI coletado corredor a corredor durante o *site survey*, que constituirá a variável de custo dinâmico das arestas nos dois motores de roteamento (IndoorAtlas, 2025).

2.3.2 A Função de Custo: Distância e RSSI

Em problemas de roteamento, cada aresta de um grafo possui um custo (ou peso) que orienta o algoritmo na escolha do melhor caminho. Na abordagem tradicional, esse custo é puramente a distância física entre dois pontos. Na modelagem deste trabalho, a Função de Custo será composta por duas variáveis complementares: a distância em metros e a qualidade do sinal Wi-Fi, medida pelo RSSI. A seguir, detalha-se como essa função será aplicada de forma distinta em cada um dos dois motores de roteamento.

Função de Custo para o Algoritmo de Dijkstra

O algoritmo de Dijkstra opera sobre um grafo com pesos **estáticos e predefinidos** nas arestas. Para compor esses pesos de forma que o roteamento favoreça corredores com melhor conectividade Wi-Fi sem ignorar completamente a distância física, este trabalho adota uma **média ponderada** como função de custo de cada aresta.

O primeiro passo da formulação é a transformação do RSSI bruto em uma métrica positiva de penalidade. Como o RSSI opera em escala negativa — valores mais próximos de zero indicam sinal melhor, enquanto valores mais negativos indicam sinal degradado (CHIA et al., 2025) — é necessário invertê-lo em um custo proporcional à degradação do sinal. Para isso, aplica-se a normalização *min-max*, técnica de pré-processamento que transforma qualquer variável numérica para o intervalo $[0, 1]$ por meio da subtração do valor mínimo dividida pela amplitude total dos dados (HAN; KAMBER; PEI, 2011). No presente trabalho, essa técnica é adaptada ao contexto de roteamento: em vez de normalizar diretamente o RSSI, inverte-se a escala para que valores de sinal degradado resultem em custo alto, e valores de sinal excelente resultem em custo baixo:

$$P_{RSSI}(u, v) = \frac{RSSI_{max} - \overline{RSSI}(u, v)}{RSSI_{max} - RSSI_{min}}$$

onde $\overline{RSSI}(u, v)$ é a média aritmética das leituras coletadas no corredor entre os nós u e v , $RSSI_{max}$ é o pior sinal registrado no *dataset* e $RSSI_{min}$ é o melhor sinal registrado. Dessa forma, corredores com sinal excelente recebem P_{RSSI} próximo de 0 (baixo custo), enquanto corredores com sinal degradado recebem P_{RSSI} próximo de 1 (alto custo).

A distância física da aresta é igualmente normalizada para que ambas as variáveis operem na mesma escala $[0, 1]$, dividindo cada distância pelo maior valor de aresta presente no grafo (HAN; KAMBER; PEI, 2011):

$$D_{norm}(u, v) = \frac{d(u, v)}{d_{max}}$$

onde $d(u, v)$ é a distância real em metros entre os nós u e v , e d_{max} é a maior distância de aresta presente no grafo.

Com ambas as variáveis normalizadas, a função de custo final da aresta é definida como uma combinação linear ponderada. Essa abordagem — denominada na literatura como *Weighted Sum-Dijkstra's Algorithm* (WSDA) — consiste em agregar múltiplos critérios normalizados por meio de pesos que refletem a prioridade de cada variável, permitindo que o algoritmo de Dijkstra opere sobre um custo composto em vez de um único critério (HUA; ABDULLAH, 2018). No presente trabalho, a formulação é adaptada para o contexto de roteamento *indoor* com dois critérios: a qualidade do sinal Wi-Fi e a distância física, com peso majoritário atribuído ao sinal, dado que o objetivo central da pesquisa é priorizar a conectividade:

$$C_{Dijkstra}(u, v) = 0,7 \cdot P_{RSSI}(u, v) + 0,3 \cdot D_{norm}(u, v)$$

A distribuição de 70% para o sinal e 30% para a distância constitui a hipótese inicial deste trabalho e será submetida a ajuste empírico durante a fase de calibração, caso os resultados preliminares indiquem necessidade de revisão.

2.3.3 Implementação do Algoritmo de Dijkstra

O algoritmo de Dijkstra será implementado como a abordagem de base (*baseline*) deste estudo. Trata-se de um algoritmo de busca não informada que garante a descoberta do caminho de menor custo entre um nó de origem e todos os outros nós de um grafo, desde que os pesos das arestas sejam não negativos (CORMEN et al., 2009).

Entrada: o Grafo Ponderado

O algoritmo receberá como entrada o grafo $G = (V, A)$ construído a partir do mapeamento topológico do Centro Universitário Senac via *IndoorAtlas*, conforme descrito na Seção 2.3.1. Cada aresta $(u, v) \in A$ terá como peso o valor $C_{Dijkstra}(u, v)$, calculado pela função de custo ponderada definida na Seção 2.3.2:

$$C_{Dijkstra}(u, v) = 0,7 \cdot P_{RSSI}(u, v) + 0,3 \cdot D_{norm}(u, v)$$

Esses pesos são **estáticos**: calculados uma única vez a partir das médias do *dataset* empírico de RSSI e fixados antes da execução do algoritmo. Essa característica é o que classifica o Dijkstra como uma abordagem determinística — ele encontra a rota ótima para um conjunto fixo de custos, mas é incapaz de reagir a variações desses custos em tempo real (PHAM et al., 2025).

Execução: o Mecanismo de Relaxamento

Durante sua execução, o algoritmo mantém duas estruturas de dados principais (CORMEN et al., 2009):

- Um conjunto S de vértices cujas distâncias mínimas a partir do nó de origem já foram definitivamente calculadas;
- Uma fila de prioridade Q contendo os vértices ainda não processados, ordenados pela menor estimativa de custo acumulado.

A cada iteração, o algoritmo extrai de Q o vértice u com menor custo acumulado, adiciona-o a S e aplica a operação de **relaxamento** sobre todas as arestas que partem de u . O relaxamento verifica se o caminho até um vértice vizinho v passando por u é mais barato do que o caminho anteriormente conhecido para v :

$$d(v) = \min(d(v), d(u) + C_{Dijkstra}(u, v))$$

onde $d(v)$ é o custo acumulado atual para alcançar o nó v e $d(u)$ é o custo acumulado para alcançar o nó atual u (CORMEN et al., 2009). O algoritmo repete esse processo até que todos os vértices tenham sido processados ou até que o nó de destino seja alcançado.

Saída e Complexidade

A saída do algoritmo será o **caminho de menor custo ponderado** entre o nó de origem e o nó de destino definidos pelo pesquisador, representando a rota que melhor equilibra conectividade Wi-Fi e distância física segundo os pesos definidos. A complexidade de tempo do algoritmo com fila de prioridade é $O((V + A) \log V)$, onde V é o número de vértices e A é o número de arestas do grafo (CORMEN et al., 2009).

Função de Custo e Ambiente de Treinamento para o *Q-Learning*

O *Q-Learning* não opera sobre pesos estáticos. Em vez disso, ele aprende o custo de cada aresta por meio da interação repetida com um **ambiente simulado estocástico**, que reproduz as flutuações reais do sinal Wi-Fi observadas nos corredores do Senac. A

construção desse ambiente é o que diferencia fundamentalmente a abordagem da IA do método determinístico.

Protocolo de coleta empírica: Para construir esse ambiente com fidelidade estatística, o *dataset* de RSSI será coletado ao longo de **três semanas**, em **três dias por semana**, com medições realizadas em **dois horários distintos** por dia: entre 18h00 e 19h00 (antes do início das aulas, com baixo fluxo de pessoas) e entre 20h50 e 21h20 (durante o intervalo, com alto fluxo de pessoas), conforme definido na Seção 1.3. Esse protocolo foi desenhado para capturar a variabilidade estocástica real do sinal descrita por Chia et al. (2025), que demonstra que fatores como a circulação de pessoas e o ruído eletromagnético causam flutuações sistemáticas e mensuráveis no RSSI de corredores internos.

Extração das métricas estatísticas: Ao final do período de coleta, para cada aresta (u, v) do grafo, serão extraídas três métricas do conjunto de leituras de RSSI registradas naquele corredor:

- $\overline{RSSI}(u, v)$: a **média aritmética** das leituras, representando o nível típico de sinal naquele trecho;
- $RSSI_{min}(u, v)$ e $RSSI_{max}(u, v)$: os valores **mínimo e máximo** observados naquele corredor, delimitando o intervalo real de variação do sinal empiricamente registrado.

Simulação do ambiente dinâmico: A cada episódio de treinamento do agente, o valor de RSSI de cada aresta não é fixo, mas sim **amostrado aleatoriamente** dentro do intervalo empiricamente observado para aquele corredor. Durante a elaboração deste trabalho, foi realizada uma pesquisa na literatura em busca de um modelo probabilístico que representasse com fidelidade a variação real do sinal RSSI em ambientes *indoor* dinâmicos. Não foi identificada, contudo, uma distribuição consensual que refletisse adequadamente esse comportamento, dado que a variação do sinal é fortemente dependente das características físicas e do fluxo de pessoas de cada ambiente específico (CHIA et al., 2025). Diante disso, optou-se por uma abordagem conservadora, construindo o ambiente de simulação exclusivamente a partir dos limites observados nos dados empíricos coletados:

$$RSSI_{sim}(u, v) \sim U(RSSI_{min}(u, v), RSSI_{max}(u, v))$$

onde:

- $RSSI_{sim}(u, v)$ é o valor de sinal simulado para a aresta entre os nós u e v em um determinado episódio de treinamento;
- $U(a, b)$ denota a distribuição uniforme contínua no intervalo $[a, b]$, garantindo que qualquer valor dentro da amplitude real de variação observada naquele corredor possa ocorrer com igual probabilidade;

- $RSSI_{min}(u, v)$ e $RSSI_{max}(u, v)$ são os valores mínimo e máximo registrados empiricamente naquele corredor durante o protocolo de coleta, delimitando o intervalo real de flutuação do sinal.

O agente é assim exposto a toda a amplitude real de variação do sinal registrada nos corredores do Senac, aprendendo a tomar decisões robustas para qualquer condição dentro desse intervalo observado, sem extrapolações além do que foi medido.

Função de recompensa: A partir do $RSSI_{sim}$ amostrado, a função de recompensa R do agente é definida como:

$$R(u, v) = \begin{cases} +10 & \text{se } RSSI_{sim}(u, v) \geq -60 \text{ dBm (sinal aceitável)} \\ -10 & \text{se } RSSI_{sim}(u, v) < -60 \text{ dBm (zona de sombra)} \\ +50 & \text{se o agente alcança o nó de destino} \end{cases}$$

Essa estrutura de recompensa força o agente a aprender, ao longo dos episódios de treinamento, quais corredores apresentam sinal estável e quais representam zonas de risco, consolidando esse conhecimento na Tabela Q (SUTTON; BARTO, 2018).

2.3.4 Implementação do Algoritmo *Q-Learning*

O *Q-Learning* será implementado como a abordagem de Inteligência Artificial deste estudo. Diferentemente do Dijkstra, que opera sobre pesos estáticos e predefinidos, o *Q-Learning* aprende a política de roteamento ótima por meio da interação iterativa com o ambiente simulado estocástico descrito na Seção 2.3.2, sem necessitar conhecer previamente os custos reais das arestas (SUTTON; BARTO, 2018).

Modelagem como Processo de Decisão de Markov

A estrutura do algoritmo baseia-se em um **Processo de Decisão de Markov** (MDP — *Markov Decision Process*), formalismo matemático que modela problemas de tomada de decisão sequencial em ambientes com incerteza (SUTTON; BARTO, 2018). No contexto deste trabalho, o MDP é definido pela seguinte tripla:

- **Estado** (S): o nó (vértice) atual em que o agente se encontra no grafo $G = (V, A)$. O espaço de estados é finito e corresponde ao conjunto completo de vértices do grafo, ou seja, todos os corredores e cruzamentos mapeados do Centro Universitário Senac;
- **Ação** (A): o movimento do agente para um nó vizinho através de uma aresta disponível. O conjunto de ações possíveis em cada estado é limitado pelos vizinhos diretos do vértice atual no grafo, garantindo que o agente só possa se mover por caminhos fisicamente existentes;

- **Recompensa (R):** o *feedback* numérico retornado pelo ambiente após cada ação, conforme a função de recompensa definida na Seção 2.3.2. Transições por corredores com sinal estável geram recompensas positivas, incentivando o agente a preferir esses trajetos; travessias por zonas de sombra (RSSI abaixo do limiar de -60 dBm) geram punições severas, ensinando o agente a evitá-las; e a chegada ao nó de destino gera uma recompensa máxima, orientando o agente em direção ao objetivo (SUTTON; BARTO, 2018).

A Tabela Q e a Equação de Bellman

O núcleo do aprendizado reside em uma estrutura matricial denominada **Tabela Q** (*Q-Table*), onde cada célula $Q(s, a)$ armazena a estimativa acumulada de recompensa futura associada a tomar a ação a a partir do estado s (SUTTON; BARTO, 2018). No início do treinamento, todos os valores da Tabela Q são inicializados em zero, representando o estado de ignorância total do agente sobre o ambiente.

A cada passo de interação com o ambiente, o agente atualiza a célula correspondente da Tabela Q utilizando a **Equação de Bellman** (SUTTON; BARTO, 2018):

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

onde cada termo possui o seguinte significado:

- $Q(S_t, A_t)$ é o valor atual registrado na Tabela Q para a ação A_t tomada no estado S_t , representando a estimativa de qualidade dessa decisão antes da atualização;
- α (*Learning Rate* — Taxa de Aprendizado, $0 < \alpha \leq 1$): controla o quanto a nova experiência substitui o conhecimento anterior. Valores altos tornam o agente mais reativo a novas informações, mas menos estável; valores baixos tornam o aprendizado mais conservador e gradual (SUTTON; BARTO, 2018);
- R_{t+1} é a recompensa imediata recebida pelo agente após executar a ação A_t no estado S_t , calculada pela função de recompensa definida na Seção 2.3.2;
- γ (*Discount Factor* — Fator de Desconto, $0 \leq \gamma \leq 1$): determina o quanto o agente valoriza recompensas futuras em relação às imediatas. Um valor de γ próximo de 1 torna o agente preocupado com o longo prazo da rota; um valor próximo de 0 torna-o míope, priorizando apenas a recompensa imediata (SUTTON; BARTO, 2018);
- $\max_a Q(S_{t+1}, a)$ é a melhor estimativa de recompensa futura disponível na Tabela Q a partir do próximo estado S_{t+1} , representando o valor da decisão ótima que o agente acredita poder tomar a seguir;
- O termo $[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$ é denominado **erro de diferença temporal** (*TD error*): mede a discrepância entre o que o agente esperava obter e o

que efetivamente obteve, sendo esse erro o sinal que conduz o aprendizado (SUTTON; BARTO, 2018).

Política de Exploração: *Epsilon-Greedy*

Durante o treinamento, o agente precisa equilibrar duas necessidades conflitantes: **explorar** novos caminhos desconhecidos (que podem ser melhores do que os já conhecidos) e **explorar** o conhecimento já acumulado na Tabela Q (escolhendo sempre a melhor ação conhecida). Para isso, será adotada a política *epsilon-greedy* (ϵ -greedy) (SUTTON; BARTO, 2018): com probabilidade ϵ o agente escolhe uma ação aleatória (exploração), e com probabilidade $1 - \epsilon$ escolhe a ação de maior valor Q conhecido (exploração). O valor de ϵ será reduzido gradualmente ao longo dos episódios de treinamento, de forma que o agente explore amplamente no início e passe a explorar o conhecimento consolidado ao final.

Fases de Execução e Papel no Comparativo

Assim como descrito na Seção 2.1.4, a execução do modelo divide-se em duas fases (PHAM et al., 2025):

- **Fase *Offline* (Treinamento):** o agente percorre o grafo em múltiplos episódios simulados, recebendo valores de RSSI amostrados dinamicamente de $U(RSSI_{min}(u, v), RSSI_{max}(u, v))$ a cada passo. Ao longo dos episódios, a Tabela Q converge para uma representação estável das qualidades de cada corredor, incorporando a amplitude real de variação do sinal observada empiricamente;
- **Fase *Online* (Inferência):** com a Tabela Q consolidada, o roteamento ocorre de forma imediata por simples consulta à matriz: o agente seleciona, em cada nó, a ação de maior valor Q disponível. Conforme demonstrado por Pham et al. (2025), apesar do tempo computacional envolvido durante a fase de treinamento *offline*, o roteamento Q pode ser executado com uma complexidade de tempo de execução equivalente à do algoritmo de Dijkstra durante a fase *online*. Essa característica é relevante no contexto deste estudo: enquanto o Dijkstra exigiria recálculo completo a cada variação de sinal Wi-Fi, a IA infere a rota a partir do aprendizado acumulado, sem necessitar conhecer os custos reais das arestas em tempo real (PHAM et al., 2025).

2.4 Trabalhos relacionados

Nesta seção, detalham-se três trabalhos que compartilham o foco direto na resolução das limitações de roteamento e posicionamento frente a ambientes estocásticos. Os estudos

selecionados validam as tecnologias e as abordagens matemáticas que fundamentam esta pesquisa, abordando a ineficácia do GPS *indoor*, a otimização de caminhos via Aprendizado por Reforço e a vulnerabilidade da métrica RSSI. A análise destas obras evidencia as lacunas de modelos estáticos e justifica a integração tecnológica executada neste TCC.

2.4.1 Trabalho 1: Indoor Atlas Service as a Tool for Building an Interior Navigation System

Štancel et al. (2021) abordaram a ineficácia do Sistema de Posicionamento Global (GPS) em ambientes fechados e a complexidade de converter a arquitetura física predial em modelos computacionais de navegação. Para a resolução deste obstáculo, os autores modelaram as plantas baixas em redes navegáveis (grafos) utilizando a plataforma *IndoorAtlas*, que explora os sensores geomagnéticos dos *smartphones*. A metodologia abrangeu a digitalização do ambiente e a execução do Algoritmo de Dijkstra. Os resultados comprovaram a viabilidade técnica da ferramenta para extrair rotas estruturadas e validaram o Dijkstra como um método de altíssima eficiência para o cálculo do caminho mínimo em redes de topologia fixa. No entanto, a solução possui uma limitação inerente: o ambiente foi tratado como estritamente estático. O peso das arestas do grafo baseou-se exclusivamente na distância física (metros). O presente trabalho diferencia-se por evoluir essa malha geométrica, substituindo o peso estático de distância por uma função de custo dinâmica baseada na variação do sinal de rede, adaptando a rota à realidade operacional do ambiente.

2.4.2 Trabalho 2: Reinforcement learning based estimation of shortest paths in dynamically changing transportation networks

Pham et al. (2025) investigaram as limitações dos algoritmos tradicionais de roteamento (como Dijkstra e A*) frente a ambientes dinâmicos e estocásticos, nos quais os custos de locomoção flutuam constantemente devido a eventos imprevistos do mundo real. A abordagem consistiu na implementação do *Q-Learning* para capacitar o sistema a descobrir rotas ótimas de forma autônoma através de interações contínuas e acúmulo de experiência. Os resultados demonstraram que a Inteligência Artificial estima as melhores rotas com a mesma exatidão matemática do Dijkstra sob condições de incerteza. A limitação deste estudo, contudo, reside em seu escopo de aplicação: o foco recaiu estritamente sobre a navegação *outdoor* em larga escala (tráfego veicular), utilizando o congestionamento de trânsito como o peso dinâmico. O presente trabalho diferencia-se por transpor a eficiência matemática do *Q-Learning* para o micromapeamento *indoor*, substituindo o tráfego de veículos pela flutuação estocástica do sinal Wi-Fi como a variável de penalidade da rede.

2.4.3 Trabalho 3: The Challenge of Dynamic Environments in Regard to RSSI-Based Indoor Wi-Fi Positioning

Chia et al. (2025) avaliaram a vulnerabilidade do sinal Wi-Fi (RSSI) em relação a fatores externos em Ambientes Dinâmicos (DE), como obstáculos físicos, ruído ambiental e interferência de múltiplos caminhos. A metodologia englobou uma Revisão Sistemática da Literatura (RSL) sobre 128 artigos acadêmicos (2018–2024) para mensurar o impacto do caos ambiental nas tecnologias de posicionamento baseadas em rádio e testar a eficácia de algoritmos de *Machine Learning* (ML) e *Deep Learning* (DL). Os resultados comprovaram que o desempenho *indoor* sofre degradação drástica devido a fatores dinâmicos de movimentação estrutural e humana, causando atenuação e reflexão. Adicionalmente, atestou-se que modelos tradicionais de ML enfrentam desafios e frequentemente exigem re-treinamento sob estas condições, exigindo o rigor do DL para contornar incertezas. A limitação da literatura analisada na RSL, no entanto, é o foco exclusivo na descoberta da coordenada isolada do usuário (*fingerprinting* de localização), sem a aplicação direta em descoberta de caminhos. O presente trabalho diferencia-se por utilizar essa degradação atestada do sinal Wi-Fi não como um problema de classificação de localização, mas sim como a matriz de custo dinâmico dentro de um algoritmo de roteamento.

A Tabela 1 sintetiza o comparativo metodológico, evidenciando como a presente pesquisa integra e avança as lacunas identificadas nestas obras basilares.

Tabela 1 – Comparação entre trabalhos relacionados e a proposta atual

Critério	Štancel et al. (2021)	Pham et al. (2025)	Chia et al. (2025)	Este trabalho
Objetivo	Mapeamento digital e navegação estrutural predial.	Descoberta do caminho mínimo em redes veiculares dinâmicas.	Avaliação do impacto de ambientes dinâmicos (DE) no sinal Wi-Fi.	Roteamento adaptativo <i>indoor</i> via estabilidade Wi-Fi.
Tecnologia	<i>IndoorAtlas</i> ; Grafo e Algoritmo de Dijkstra.	Aprendizado por Reforço (<i>Q-Learning</i>); Dijkstra; A*.	Modelos de <i>Machine Learning</i> (ML) e <i>Deep Learning</i> (DL).	Dijkstra vs <i>Q-Learning</i> ; RSSI via <i>IndoorAtlas</i> .
Validação	Medições empíricas em campo do desvio de trajetória (erro em centímetros).	Microsimulação de tráfego virtual (VISSIM) e comparação de tempo.	Revisão Sistemática da Literatura (análise de métricas de 128 artigos).	Testes comparativos empíricos em cenários de controle e estresse estocástico.
Limitação	Omissão de interferências dinâmicas no cálculo estático da rota.	Escopo focado estritamente em tráfego <i>outdoor</i> e macro-logística.	Ausência de aplicação direta da métrica em motores de roteamento.	—

Fonte: Elaborado pelo autor (2026)

3 Desenvolvimento

Neste capítulo, apresenta-se a solução proposta para o problema identificado, bem como o cronograma de trabalho previsto para o desenvolvimento completo do projeto no TCC2.

3.1 Solução proposta

A solução compreende a estruturação de um ambiente computacional de simulação e roteamento empírico. O escopo do projeto não visa a entrega de um aplicativo ou produto comercial para o usuário final, mas sim a construção de uma bancada de testes algorítmica. Este ambiente consumirá dados topológicos e de radiofrequência reais, modelará o edifício do Centro Universitário Senac em uma malha de grafos e executará dois motores de roteamento (Dijkstra e *Q-Learning*) puramente para analisar e comparar suas capacidades de contornar áreas de alta atenuação de sinal Wi-Fi.

A seguir, detalham-se os pilares que compõem a abordagem:

- **Visão geral da solução:** A entrega deste trabalho consiste em um ambiente computacional de simulação construído para executar e comparar dois algoritmos de roteamento sob condições reais de variação de sinal Wi-Fi. O ambiente consumirá dados topológicos e de radiofrequência coletados empiricamente nos corredores do Centro Universitário Senac, modelará essa estrutura física como um grafo ponderado e submeterá os dois motores de roteamento a cenários controlados de teste, extraindo métricas comparativas de desempenho.
- **Arquitetura:** O ambiente de simulação é composto por quatro módulos interdependentes, executados sequencialmente:
 1. **Módulo de Coleta e Mapeamento:** Utilizando a plataforma *IndoorAtlas*, serão realizadas coletas presenciais nos corredores do Senac ao longo de três semanas, conforme o protocolo definido na Seção 2.3.2. Esse módulo produz duas saídas: a topologia do edifício convertida em grafo $G = (V, A)$ com coordenadas geográficas reais, e o *dataset* empírico de RSSI por corredor, contendo as leituras brutas coletadas nos dois horários de coleta.
 2. **Módulo de Estruturação Lógica:** Os dados brutos de RSSI passam por tratamento estatístico: remoção de leituras discrepantes, cálculo da média $\mu(u, v)$ e identificação dos valores mínimo $RSSI_{min}(u, v)$ e máximo $RSSI_{max}(u, v)$ por aresta, e aplicação da normalização min-max para conversão em penali-

dade $P_{RSSI}(u, v)$. Ao final desse módulo, cada aresta do grafo possui seu peso $C_{Dijkstra}(u, v)$ calculado e pronto para consumo pelos motores de roteamento.

3. **Motor Determinístico — Dijkstra:** Recebe o grafo com pesos estáticos e executa o algoritmo de Dijkstra conforme descrito na Seção 2.3.3, calculando a rota de menor custo ponderado entre a origem e o destino definidos. Opera com os valores fixos de $\mu(u, v)$ nas arestas, sem capacidade de reagir a variações do sinal durante a execução.
 4. **Motor Estocástico — *Q-Learning*:** Recebe o grafo e os limites empíricos $RSSI_{min}(u, v)$ e $RSSI_{max}(u, v)$ de cada aresta. Durante a fase de treinamento, simula um Ambiente Dinâmico amostrando os valores de RSSI a partir de $U(RSSI_{min}(u, v), RSSI_{max}(u, v))$ a cada episódio, conforme descrito na Seção 2.3.2. Após a convergência da Tabela Q, executa a fase de inferência para os mesmos pares origem-destino utilizados pelo Dijkstra.
- **Tecnologias e ferramentas:** A linguagem adotada para a implementação da simulação será o **Python**. A escolha é respaldada por sua consolidação como a linguagem mais utilizada em computação científica, ciência de dados e Inteligência Artificial, sendo a única que combina alta legibilidade com acesso direto a bibliotecas de baixo nível de alto desempenho (RASCHKA; PATTERSON; NOLET, 2020). Essa característica é determinante para este trabalho, que exige tanto a manipulação matricial da Tabela Q quanto o processamento estatístico do *dataset* de RSSI em um único ambiente de desenvolvimento.

Para a representação e varredura do grafo $G = (V, A)$, será utilizada a biblioteca **NetworkX** (HAGBERG; SCHULT; SWART, 2008), que fornece estruturas de dados nativas para grafos ponderados e implementações prontas de algoritmos de caminhos mínimos, incluindo o próprio Dijkstra. O processamento estatístico do *dataset* de RSSI — cálculo de médias, desvios padrão e remoção de *outliers* — será conduzido com as bibliotecas **Pandas** e **NumPy**, componentes centrais do ecossistema científico do Python (RASCHKA; PATTERSON; NOLET, 2020).

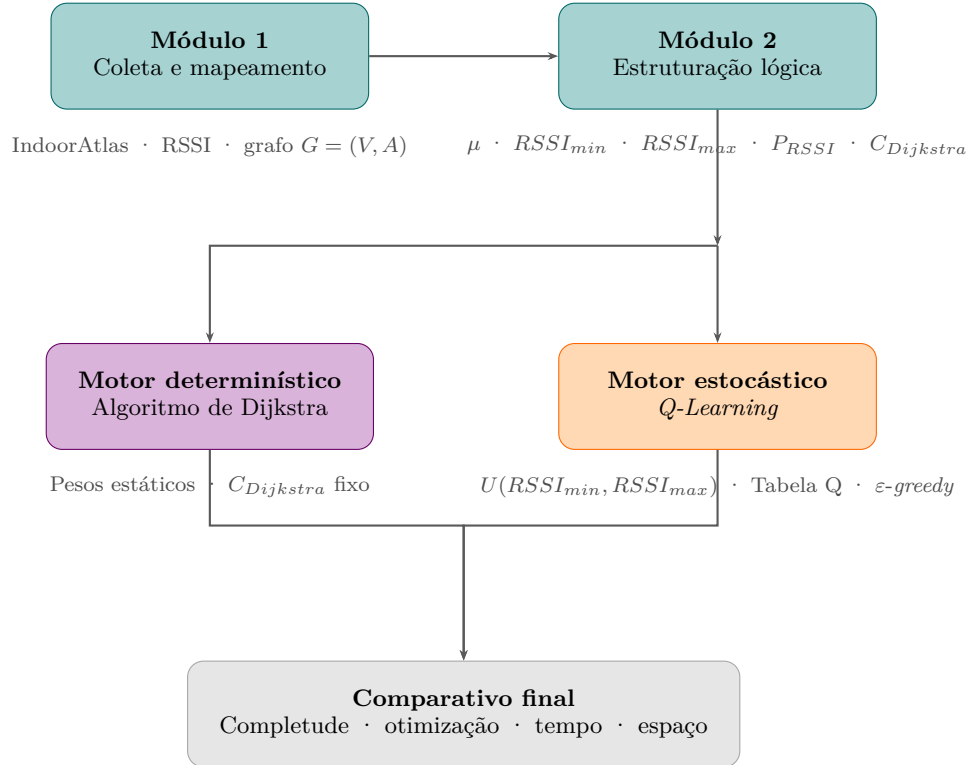
Para a obtenção da malha física e dos dados brutos de RSSI, será utilizada a **IndoorAtlas REST API** (IndoorAtlas, 2025). O emprego desta plataforma como base de mapeamento e coleta possui validação técnica estabelecida por Štancel et al. (2021) e, no cenário nacional, por Mendes et al. (2016), que atestaram sua eficácia na extração de assinaturas de rádio sem a necessidade de instalação de hardware adicional na infraestrutura do ambiente estudado.

- **Funcionalidades principais:** Para atender aos objetivos propostos e solucionar o problema de locomoção *indoor* mantendo conectividade Wi-Fi, a solução executará as seguintes funções:

- O mapeamento da topologia e a coleta contínua da potência do sinal Wi-Fi (RSSI) nos corredores do Senac via *IndoorAtlas* ao longo de um período de tempo três semanas;
 - O cálculo da rota ideal pelo algoritmo de Dijkstra, utilizando como peso das arestas a distância em metros atrelada a uma consolidação matemática do sinal RSSI;
 - A modelagem do ambiente de Aprendizado por Reforço e o treinamento do algoritmo de *Q-Learning* (fornecendo o grafo, os estados, as ações e a função de recompensa);
 - A predição autônoma da melhor rota possível pela Inteligência Artificial após a consolidação do seu aprendizado;
 - A geração de um comparativo final de sucesso e eficiência entre a abordagem determinística (Dijkstra) e a IA.
- **Metodologia de desenvolvimento:** A implementação seguirá uma abordagem experimental e iterativa, organizada em três etapas sequenciais. Na primeira, será realizada a coleta presencial do RSSI nos corredores do Senac, conforme o protocolo de duas semanas definido na Seção 2.3.2. A viabilidade dessa coleta na instituição possui embasamento prévio comprovado por [Claro \(2022\)](#), que realizou levantamento de impressões digitais de rádio no mesmo ambiente. Na segunda etapa, os dados brutos serão tratados estatisticamente e o grafo ponderado será construído conforme descrito nos Módulos 1 e 2 da arquitetura. Na terceira, os dois algoritmos serão implementados e submetidos a ciclos de ajuste paramétrico — calibrando a taxa de aprendizado α e o fator de desconto γ do *Q-Learning* — até a estabilização dos resultados.
 - **Critérios de validação:** A eficácia comparativa será avaliada por meio de uma bateria de testes controlada, submetendo os dois motores de roteamento a dois cenários distintos: o **Cenário de Controle**, no qual os valores de RSSI amostrados refletem condições estáveis de sinal, e o **Cenário de Estresse**, no qual os valores são amostrados a partir dos piores quartis do *dataset*, simulando degradações severas de conectividade. Com base nos critérios de avaliação de algoritmos de roteamento adotados por [Pham et al. \(2025\)](#), o desempenho de cada abordagem será medido por quatro métricas: *Compleitude* (capacidade de encontrar uma rota válida em todos os cenários), *Grau de Otimização* (qualidade da rota encontrada em termos de conectividade e distância), *Complexidade de Tempo* (custo computacional de execução, expresso em notação O) e *Complexidade de Espaço* (memória alocada durante a execução).

Para ilustrar a estrutura macro de processamento e execução da simulação, a Figura 7 apresenta o fluxo arquitetural da solução.

Figura 7 – Fluxo do Ambiente Computacional de Simulação

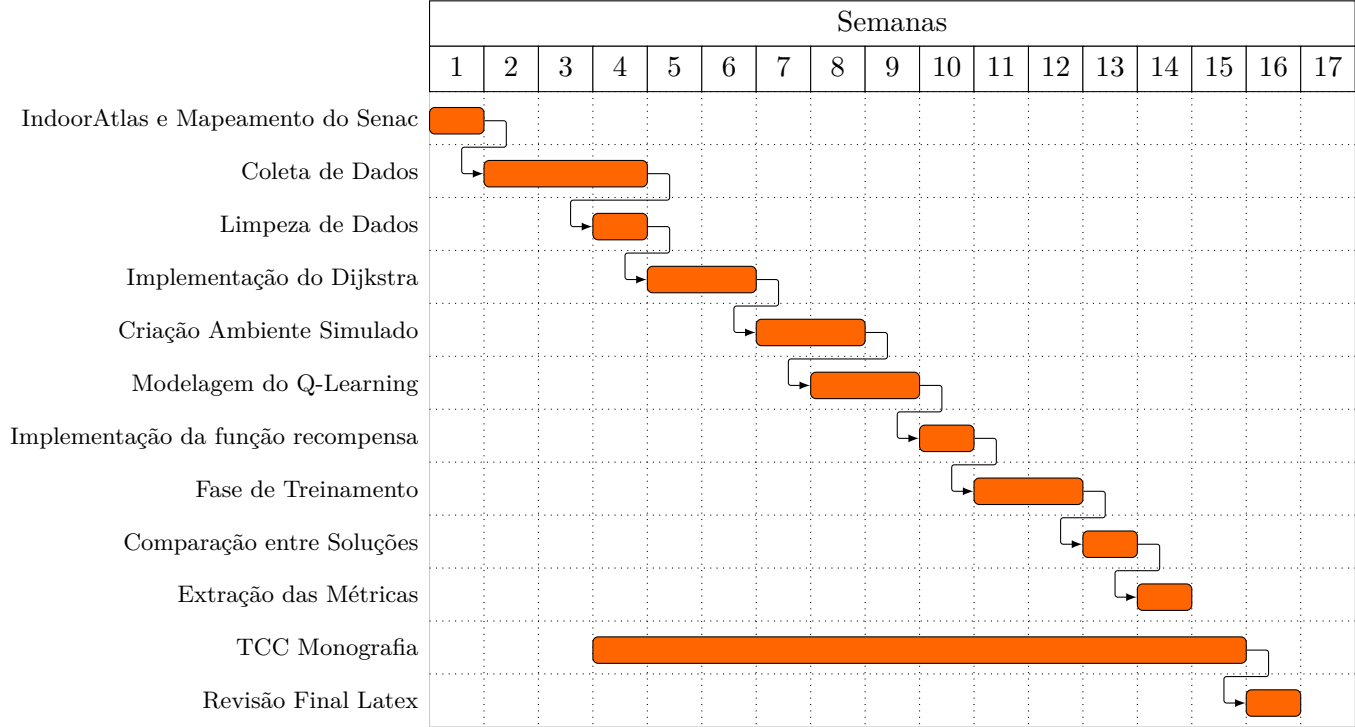


Fonte: Elaborado pelo autor (2026)

3.2 Cronograma de trabalho

O cronograma a seguir apresenta, no formato de diagrama de Gantt, as atividades previstas para o desenvolvimento do TCC2, distribuídas ao longo das semanas do semestre.

Figura 8 – Cronograma de atividades – Diagrama de Gantt



Fonte: Elaborado pelo autor (2026)

Referências

BARROSO, G. R. L.; BRITO, P. H. O. *Protótipo funcional de localização indoor baseado em grafos e algoritmos de busca*. Dissertação (Trabalho de Conclusão de Curso (Graduação em Ciência da Computação)) — Centro Universitário Senac, São Paulo, 2025. 8, 13

BOYAN, J. A.; LITTMAN, M. L. Packet routing in dynamically changing networks: A reinforcement learning approach. In: *Advances in Neural Information Processing Systems*. [S.l.]: Morgan Kaufmann, 1993. v. 6, p. 671–678. 15

CHIA, Z. Y. et al. The challenge of dynamic environments in regard to RSSI-based indoor Wi-Fi positioning — a systematic review. *Future Internet*, v. 17, n. 12, p. 540, 2025. 8, 9, 10, 12, 13, 19, 22, 27

CLARO, L. B. R. *Comparação de desempenho dos algoritmos KNN e Bayes Ingênuo para localização interna com fingerprinting*. Dissertação (Trabalho de Conclusão de Curso (Graduação em Ciência da Computação)) — Centro Universitário Senac, São Paulo, 2022. 31

CORMEN, T. H. et al. *Introduction to Algorithms*. 3. ed. Cambridge: MIT Press, 2009. 13, 20, 21

GOLDBARG, M. C.; GOLDBARG, E. *Grafos: Conceitos, Algoritmos e Aplicações*. Rio de Janeiro: Elsevier, 2012. 13

HAGBERG, A. A.; SCHULT, D. A.; SWART, P. J. Exploring network structure, dynamics, and function using NetworkX. In: *Proceedings of the 7th Python in Science Conference*. Pasadena, CA, USA: [s.n.], 2008. p. 11–15. 30

HAN, J.; KAMBER, M.; PEI, J. *Data Mining: Concepts and Techniques*. 3. ed. Burlington: Morgan Kaufmann, 2011. 19, 20

HUA, T. K.; ABDULLAH, N. Weighted sum-dijkstra's algorithm in best path identification based on multiple criteria. *Journal of Physics: Conference Series*, v. 1132, p. 012003, 2018. 20

IndoorAtlas. *Positioning REST API Overview*. 2025. Acesso em: 05 jun. 2026. Disponível em: <<https://support.indooratlas.com/support/solutions/articles/36000051260-positioning-rest-api-overview>>. 18, 19, 30

Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira. *Exame Nacional de Desempenho dos Estudantes (ENADE) 2021 — Ciência da Computação (Bacharelado), Questão 34*. 2021. Acesso em: 06 jun. 2026. Disponível em: <<https://www.questoes.indagacao.com/2021/12/o-algoritmo-de-dijkstra-para-o-problema-do-caminho-minimo-em-digrafos-com-pesos-utiliza-uma-fila.html>>. 14

MENDES, N. G. et al. Sistema de posicionamento local IndoorAtlas aplicado no Instituto Federal Farroupilha Campus São Borja. In: *Anais do EATI – Encontro Anual de Tecnologia da Informação e STIN – Simpósio de Tecnologia da Informação da Região Noroeste do RS*. Frederico Westphalen, RS: [s.n.], 2016. v. 6, n. 1, p. 101–107. 30

- PHAM, H. D. et al. Reinforcement learning based estimation of shortest paths in dynamically changing transportation networks. *Frontiers in Future Transportation*, v. 6, 2025. [8](#), [9](#), [10](#), [14](#), [15](#), [21](#), [25](#), [26](#), [31](#)
- RASCHKA, S.; PATTERSON, J.; NOLET, C. Machine learning in Python: Main developments and technology trends in data science, machine learning, and artificial intelligence. *Information*, v. 11, n. 4, p. 193, 2020. [30](#)
- SUTTON, R. S.; BARTO, A. G. *Reinforcement Learning: An Introduction*. 2. ed. Cambridge: MIT Press, 2018. [14](#), [15](#), [23](#), [24](#), [25](#)
- WATKINS, C. J. C. H. *Learning from delayed rewards*. Tese (Tese de Doutorado) — King's College, Cambridge, Cambridge, Inglaterra, 1989. [15](#)
- ŠTANCEL, M. et al. Indoor Atlas service as a tool for building an interior navigation system. *IFAC-PapersOnLine*, v. 18, n. 9, 2021. [8](#), [9](#), [12](#), [13](#), [17](#), [18](#), [19](#), [26](#), [30](#)